



nettest Administrator Guide

Network path troubleshooting with the nettest client and server

Version 1.1.0 · Covers Windows 11 and Linux · October 2026 (revision 3: endpoint monitoring service with ntfy / Slack / Discord alerts)

nettest is a two-part tool for diagnosing flaky network paths. The **server** runs at the far end and echoes test traffic. The **client** runs where the problem is seen, measures round-trip latency, packet loss, jitter, connection stability and throughput, and shows the results live in a terminal window or writes them to files for a ticket. Both parts are single executables with no installer and no runtime dependencies.

It speaks the same test protocol over four transports, **WebSocket (ws)**, **WebSocket over TLS (wss)**, **raw TCP** and **UDP**, so you can tell apart "the network drops packets" from "the proxy kills WebSocket connections" from "TLS negotiation is slow" using one tool. For devices that cannot run software, such as IP phones, switches and gateways, the client also probes with **ICMP ping**, **TCP connect**, **SIP OPTIONS** and **HTTP(S)**, with the same statistics, chart and reports (section 8).

New in 1.1.0: the client can run as a **monitoring service** that watches any number of endpoints around the clock and sends an **ntfy**, **Slack** or **Discord** message when one goes down and when it comes back (section 9).

1. Requirements and deployment
2. Server: settings and operation
3. Server: running as a system service
4. Client: interactive terminal UI
5. Client: settings reference
6. Client: headless and scripted use
7. Test modes and what they measure
8. Testing devices without a server
9. Client: monitoring endpoints as a service
10. Exports and log files
11. Reading the results
12. Troubleshooting recipes
13. Quick reference card

1. Requirements and deployment

Item	Requirement
Operating system	Windows 11 or later (x64). Linux x64 with a current glibc (any mainstream distribution from the last few years).
Dependencies	None. No .NET, no Visual C++ runtime, no OpenSSL, no Python. TLS is built in, including the public root certificates the monitor needs for HTTPS webhooks.
Privileges	Standard user for all tests. <code>ping://</code> uses the unprivileged ICMP API on Windows and the unprivileged "ping socket" on Linux (allowed for everyone on current Fedora, Ubuntu, Debian and other systemd-based distributions; otherwise root). Installing the server or the client monitor as a service needs administrator / root; the installer asks for it itself (sections 3 and 9).
Disk and memory	About 3 MB per executable. Memory stays flat during multi-hour runs and for a monitor that runs for months.
Terminal	Any modern terminal: Windows Terminal, PowerShell, cmd.exe, or any Linux terminal. The interactive view needs at least 80x24 characters; 130x40 is comfortable.

Network	Server: inbound access on the ports you enable (defaults below). Client: outbound access to those ports, or to the device's ICMP / SIP / HTTP ports for serverless probes. Monitor: additionally outbound HTTPS (port 443) to ntfy.sh, Slack or Discord, or to your own ntfy server. HTTP proxies are not traversed automatically; test the direct path.
Clock sync	Not required. All latency is round-trip and measured on the client's own clock.

Files to deploy

File	Where
<code>nettest-server.exe</code> / <code>nettest-server</code>	The machine at the far end of the path under test (a server in the datacenter, a cloud VM, the other office).
<code>nettest-client.exe</code> / <code>nettest-client</code>	The machine where the problem is experienced (a user's PC, a branch workstation, a kiosk). The same file is the monitor: put it on a machine that stays on, in the network whose view of the endpoints you care about.

Copy the file anywhere and run it from a terminal. On Linux mark it executable first: `chmod +x nettest-server`. For a permanent server use `nettest-server service install` (section 3), for a permanent monitor `nettest-client service install` (section 9); both copy the file to the system location and register it to start at boot. When run by hand, each side stores its saved settings and, for the server, its TLS certificate, under the per-user configuration directory:

Platform	Configuration directory
Windows	<code>%APPDATA%\nettest\</code> (for example <code>C:\Users\jdoe\AppData\Roaming\nettest\</code>)
Linux	<code>~/.config/nettest/</code>

Firewall. Windows Defender Firewall prompts the first time the server starts; allow it on the network profile you are testing (usually Private or Domain). On Linux, open the ports in `firewalld`, `ufw` or `nftables`. UDP must be allowed inbound on the server like any UDP service. The client and the monitor need no inbound rules.

2. Server: settings and operation

Start the server with no arguments for the interactive view, or with `--no-tui` to log to the console (for services, scheduled tasks and SSH sessions). Every setting is available as a command-line switch, as an environment variable, or in the saved configuration file. Precedence is switch, then environment variable, then configuration file, then built-in default.

```
nettest-server
nettest-server --no-tui --token s3cret --log-file C:\logs\nettest-server.log
nettest-server --bind 172.16.89.25 --ws-port 9100 --tcp-port 0 --udp-port 0 --wss-port 0
```

Switch	Environment	Default	Meaning
<code>--bind</code> <ip any>	NETTEST_BIND	any	Address to listen on. <code>any</code> binds both IPv4 and IPv6 wildcards. Give a specific IP to restrict to one interface.
<code>--tcp-port</code> N	NETTEST_TCP_PORT	9100	Raw TCP listener. <code>0</code> disables it.
<code>--udp-port</code> N	NETTEST_UDP_PORT	9100	UDP listener. <code>0</code> disables it.
<code>--ws-port</code> N	NETTEST_WS_PORT	9101	WebSocket (<code>ws://</code>) listener. <code>0</code> disables it.
<code>--wss-port</code> N	NETTEST_WSS_PORT	9102	WebSocket over TLS (<code>wss://</code>) listener. <code>0</code> disables it and skips certificate generation.
<code>--token</code> <secret>	NETTEST_TOKEN	(none)	Pre-shared token. Clients that do not present the same token are rejected. A foreground server without a token logs a warning at start; a service cannot be installed without one.
<code>--log-file</code> <path>		(none)	Append human-readable log lines (connects, disconnects, rejections) to this file.
<code>--jsonl</code> <path>		(none)	Append the same events as JSON Lines for log collectors.
<code>--cert-dir</code> <dir>		config dir /server/	Where <code>cert.pem</code> and <code>key.pem</code> live. Generated on first start if missing; replace with your own PEM pair if you want a CA-signed certificate.
<code>--idle-timeout</code> N		60	Seconds without traffic before a client session is closed. Must be longer than the client's heartbeat interval in soak tests.
<code>--banner</code> <text>	NETTEST_BANNER	nettest by Slaytons Technology Services	Identification text returned to port scanners and browsers on every listener (see "Scanner identification" below). The server version is appended automatically. It cannot be disabled.
<code>--no-tui</code>		off	Plain console logging instead of the interactive view.
<code>--config</code> <path>		config dir /server.toml	Use a different configuration file.

-- save-config			Write the effective settings to the configuration file, then keep running. Afterwards <code>nettest-server</code> alone starts with those settings.
----------------	--	--	---

Interactive view

The server view shows the active listeners, whether a token is required, the wss certificate fingerprint, a live table of connected clients (protocol, peer address, client id, test mode, connection age, idle time, probe count, bytes in and out) and a scrolling log.

Key	Action
<code>c</code>	Clear the log panel
<code>w</code>	Save the effective settings to the configuration file
<code>?</code>	Help overlay
<code>q</code> or <code>Ctrl-C</code>	Stop the server and exit

TLS (wss) and the fingerprint

When the wss listener is enabled the server creates a self-signed certificate on first start and prints its SHA-256 fingerprint in the log and in the view, for example:

```
wss certificate fingerprint (SHA-256):
6A:52:9B:B2:36:96:47:06:FB:F0:AC:7A:EE:A2:A7:CF:49:9D:89:45:D9:1C:9B:FE:3A:51:BD:52:21:9F:67:ED
```

Give that string to client operators. The client pins it with `--fingerprint` (or the "TLS: pinned SHA-256" field), which proves the client reached the right server even without a CA. The alternative, `--insecure`, accepts any certificate and should only be used on a trusted test network.

Scanner identification

Security scanners (nmap, Nessus, Qualys and the like) must be able to identify every listening port. Anything that connects to a nettest listener without speaking the nettest protocol therefore receives a readable answer:

What connects	What it receives
An HTTP request on the TCP, ws or wss port (a browser, <code>curl</code> , an HTTP scanner)	<code>HTTP/1.1 200 OK</code> with a <code>text/plain</code> body: <code>nettest by Slaytons Technology Services (nettest-server 1.1.0)</code>
Any other bytes, or a connection that sends nothing for 3 seconds (nmap's "null probe")	The same line without HTTP framing, then the connection is closed
A UDP datagram that is not a nettest frame	The same line as one datagram, at most once per source address every 2 seconds and 50 per second overall, so the server cannot be abused to amplify traffic
A TLS client on the wss port that never completes the handshake	The server certificate, which already identifies the service

Real clients are not affected: the first bytes are inspected and then handed to the protocol handler unchanged. Every banner reply is logged as `[scan] <protocol> <peer>: banner sent`, which also makes scan activity visible. To check a server quickly: `curl http://<host>:9100/`.

Security notes

With a token set, unauthenticated TCP and WebSocket clients receive an error and are disconnected; unauthenticated UDP packets are silently dropped, so the server cannot be used as a traffic reflector. The banner reply is the one exception and is rate-limited as described above.

A server installed as a service must have a token. Use `--generate-token` to create one and distribute it to client operators; set it on clients through `NETTEST_TOKEN` rather than on the command line where possible.

The server never executes anything from a client and exposes no files; it only echoes test frames and counts bytes.

Server exit codes: 0 clean shutdown, 1 interactive view error, 2 could not bind a port, 3 bad arguments or configuration.

3. Server: running as a system service

For a permanent test endpoint the server installs itself as a **systemd unit** on Linux or a **Windows service**, starting at boot with fixed, pre-determined settings. All of this is done by the executable itself; no unit files or `sc.exe` commands need to be written by hand.

```
nettest-server service install --token <secret>                # all four listeners,
default ports
nettest-server service install --generate-token --ws-port 9100 --tcp-port 0 --udp-port 0 --wss-
port 0
nettest-server service edit --idle-timeout 120 --wss-port 9102    # change settings and
restart
nettest-server service status
nettest-server service stop | start | restart
nettest-server service uninstall    # add --purge to delete the server's settings,
certificates and logs too
```

Subcommand	What it does
<code>install [settings]</code>	Copies the executable to the system location, writes the settings file, registers the service to start at boot and starts it. Accepts every server switch from section 2 plus <code>--generate-token</code> , <code>--no-copy</code> (run from the current location instead of copying) and, on Windows, <code>--account</code> . Re-running <code>install</code> replaces the settings but keeps an existing token unless a new one is given.
<code>edit [settings]</code>	Applies the given switches to the stored settings and restarts the service. Only the switches you give change. The token can be replaced (<code>--token</code> , <code>--generate-token</code>) but not removed.
<code>uninstall [--purge]</code>	Stops and removes the service and the copied executable. <code>--purge</code> also deletes the server's settings, certificates and log files, and nothing else: a client monitor installed on the same machine (section 9) keeps its files.
<code>start, stop, restart</code>	Control the installed service.
<code>status</code>	Shows whether the service is installed and running, the stored settings (token masked), the wss fingerprint and where the log is. Needs no elevation.

A token is mandatory. `install` refuses to continue (exit code 3) when no token is given through `--token`, the `NETTEST_TOKEN` environment variable or `--generate-token`. A generated token is printed exactly once, marked "copy it now"; store it in your password manager and give it to client operators. `edit` accepts a new token but never an empty one, so an installed service can never be opened up by mistake.

Administrator rights

Every subcommand except `status` needs root or administrator rights, and the installer obtains them itself. On **Linux** it re-runs through `sudo` (or `pkexec` / `doas` when `sudo` is absent), so the password prompt and all output appear in the same terminal. On **Windows** it triggers the User Account Control prompt; the elevated run happens in the background and its output is shown in your window when it finishes. Declining the prompt ends the command with exit code 3. Running from an already elevated shell skips the prompt.

Where things live

	Linux (systemd)	Windows
Executable	<code>/usr/local/bin/nettest-server</code>	<code>%ProgramFiles%\nettest\nettest-server.exe</code>

Settings	/etc/nettest/server.toml (root only, mode 0600)	%ProgramData%\nettest\server.toml
Certificates	/var/lib/nettest/	%ProgramData%\nettest\certs\
Log	/var/log/nettest/server.log and journalctl -u nettest-server	%ProgramData%\nettest\server.log
Registration	/etc/systemd/system/nettest-server.service	Service nettest-server ("nettest server" in Services), automatic start, depends on Tcpip
Runs as	A dynamic system user with no shell or home (DynamicUser=yes), read-only system, no privilege escalation. A port below 1024 adds only the bind-privileged-port capability.	LocalSystem by default. --account "NT AUTHORITY\NetworkService" selects a lower-privileged account; grant it write access to %ProgramData%\nettest first.

The service reads only its own settings file; the per-user %APPDATA% / ~/.config files play no part. Settings not given at install time use the defaults from section 2.

Linux: the settings file reaches the service as a systemd *credential*, which is what lets a root-only file be read by the dynamic user. This requires systemd 247 (released 2020) or newer; every supported distribution ships a newer one. systemctl stop and server shutdown are graceful.

Windows: the firewall rule is **not** created automatically. Allow nettest-server.exe inbound for the listener ports in Windows Defender Firewall (the prompt does not appear for a service). %ProgramData% is readable by local users; keep the token out of other locations.

Upgrading: copy the new executable next to the old one and run service install again with no settings; it replaces the binary, keeps the stored settings and token, and restarts.

Typical output of service status:

```

service    nettest-server.service: active
config     /etc/nettest/server.toml
bind       any
listeners  tcp 9100  udp 9100  ws 9101  wss 9102
token      set (32 chars)
cert dir   /var/lib/nettest
log file   /var/log/nettest/server.log
idle       60 s
banner     nettest by Slaytons Technology Services (nettest-server 1.1.0)
wss SHA-256
6A:52:9B:B2:36:96:47:06:FB:F0:AC:7A:EE:A2:A7:CF:49:9D:89:45:D9:1C:9B:FE:3A:51:BD:52:21:9F:67:ED

```

4. Client: interactive terminal UI

Run `nettest-client` with no arguments, or with a target URL to pre-fill the form, for example `nettest-client ws://10.0.0.5:9101`. The client moves through three screens:

```
Settings  --(s)-->  Running: stats | live chart | log  --(x)-->  Summary
```

Settings screen

All test settings are edited here. Text fields are edited with `Enter` (type, then `Enter` to commit or `Esc` to cancel). Toggles flip with `Enter` or `Space`; choice fields such as protocol cycle through their options. Invalid values are refused with a message on the status line. Fields that have no effect with the chosen protocol or mode (for example the token with `ping`, or the throughput fields in latency mode) are shown dimmed and their hint says so. The field reference is in [section 5](#).

Running screen

Stats panel (left): sent/received, lost with percentage, late/out-of-order/duplicate counts, in-flight probes, last RTT, min/avg/max, p50/p95/p99, jitter. Soak runs add disconnects, reconnects, uptime, longest outage and session length. Throughput runs show instantaneous and per-direction Mbit/s.

Chart (right): RTT over time as a line, with red markers along the baseline where probes were lost. In throughput mode it plots instantaneous Mbit/s. Keys `1` `2` `3` switch the window between the last 60 seconds, the last 5 minutes and the whole run.

Log panel (bottom): connection events, every failed probe, reconnect attempts and errors. With "Log failures only" off it also lists every successful probe.

Header: target, mode, link state (connected, disconnected, reconnecting #n; for serverless probes "disconnected" means an outage is open) and elapsed time.

Summary screen

Shown when the run ends. It lists the final statistics, connection-phase timings, stability events, and the path of the HTML report if one was written. From here you can run again with the same settings or go back to the settings screen.

Keys

Key	Screen	Action
<code>↑</code> <code>↓</code> <code>Tab</code> <code>Shift-Tab</code>	Settings	Move between fields
<code>Enter</code>	Settings	Edit a text field, or flip a toggle
<code>Space</code>	Settings	Cycle protocol, test mode, throughput direction; flip toggles
<code>s</code> or <code>F5</code>	Settings, Summary	Start the test
<code>w</code>	Settings	Save settings so they are pre-filled next time
<code>x</code> <code>s</code> <code>Esc</code>	Running	Stop the test (waits briefly for in-flight probes so loss is accurate)
<code>r</code>	Running, Summary	Write an HTML report now, from the data collected so far
<code>1</code> <code>2</code> <code>3</code>	Running, Summary	Chart window: last 60 s, last 5 min, everything
<code>c</code>	Running	Clear the log panel
<code>Enter</code> <code>Esc</code> <code>b</code>	Summary	Back to the settings screen

<code>?</code>	Anywhere	Help overlay (any key closes it)
<code>q</code> or <code>Ctrl-C</code>	Anywhere	Quit; a running test is stopped and its report written first

5. Client: settings reference

Each row is one field on the settings screen together with its command-line switch. Switches override saved settings; the saved file is `client.toml` in the configuration directory.

Field	Switch	Default	Meaning
Target host	<code>--host</code> or URL	127.0.0.1	Hostname or IP of the nettest-server, or of any device for the serverless protocols. IPv6 addresses without brackets in the form; with brackets in a URL.
Port	<code>--port</code> or URL	9100	Server port for the chosen protocol; for <code>connect</code> , <code>sip</code> , <code>http</code> , <code>https</code> the device port. Not used by <code>ping</code> . A URL without a port uses the scheme's default (ws 9101, wss 9102, tcp/udp 9100, sip 5060, http 80, https 443).
Protocol	<code>-P</code> , <code>--protocol</code> or URL scheme	ws	Against nettest-server: <code>ws</code> , <code>wss</code> , <code>tcp</code> , <code>udp</code> . Against any device: <code>ping</code> , <code>connect</code> , <code>sip</code> , <code>http</code> , <code>https</code> (section 8).
Test mode	<code>-m</code> , <code>--mode</code>	latency	<code>latency</code> , <code>soak</code> or <code>throughput</code> . See section 7 . Throughput needs a nettest-server; it is refused for the serverless protocols.
Probe interval (ms)	<code>-i</code> , <code>--interval</code>	1000	Time between probes in latency mode. Switch accepts units: <code>200ms</code> , <code>1s</code> . Minimum 1 ms.
Frame size (bytes)	<code>-s</code> , <code>--size</code>	64	Size of each probe on the wire including the 32-byte header. Keep at or below 1400 on UDP to avoid fragmentation. For <code>ping</code> it is the ICMP message size (32 to 65507); the other serverless probes ignore it.
Duration (s)	<code>-d</code> , <code>--duration</code>	0	Stop after this long; 0 means run until stopped. Switch accepts <code>30s</code> , <code>5m</code> , <code>8h</code> .
Probe count	<code>-c</code> , <code>--count</code>	0	Stop after this many probes; 0 means unlimited.
Auth token	<code>--token</code>	(none)	Must equal the server's token when one is configured.
TLS: accept any cert	<code>--insecure</code>	off	wss only. Skip certificate checking. Trusted networks only.
TLS: pinned SHA-256	<code>--fingerprint</code>	(none)	wss only. The server's certificate fingerprint; colons optional. Preferred over "accept any".
Heartbeat (ms)	<code>--heartbeat</code>	5000	Soak mode cadence (also the probe interval for serverless soak tests). Set below any suspected idle timeout to keep the connection alive, or above it to provoke the drop.
Outage after N failures	<code>--outage-after</code>	3	Serverless soak only. This many consecutive failed or unanswered probes open an outage; the next answered probe closes it.
Throughput seconds	<code>--tp-secs</code>	10	Seconds to stream in each direction.
Throughput chunk (bytes)	<code>--tp-chunk</code>	65536	Bytes per throughput frame. Capped to 1368 on UDP automatically.
Throughput direction	<code>--tp-direction</code>	both	<code>upload</code> , <code>download</code> or <code>both</code> (upload first, then download).
Log: CSV file	<code>--csv [path]</code>	off	Write a CSV log. See section 10 .

Log: text file	<code>--text [path]</code>	off	Write a plain-text log.
Log: JSON lines	<code>--jsonl [path]</code>	off	Write a JSON Lines log.
HTML report at end	<code>--report [path]</code> <code>/ --no-report</code>	on	Write a single-file HTML report when the run ends.
Log failures only	<code>--all-samples</code> turns it off	on	When on, only failed probes are listed in the log panel, console and text log. CSV and JSONL always contain every probe.
Output directory	<code>-o, --out-dir</code>	.	Where generated files go. Created if missing.
(switch only)	<code>--timeout</code>	5s	Connect timeout per attempt.
(switch only)	<code>-4 / -6</code>	auto	Force IPv4 or IPv6 when the host name resolves to both.
(switch only)	<code>--ws-path, --path</code>	/	Request path for WebSocket (useful behind a path-routing proxy) and for <code>http/https</code> probes. A path in the target URL sets it too.
(switch only)	<code>--quiet</code>	off	No console sample or event lines (headless).
(switch only)	<code>--config <path></code>		Use a different settings file.
(switch only)	<code>--save-config</code>		Save the effective settings before running.

Environment variables `NETTEST_HOST`, `NETTEST_PORT`, `NETTEST_PROTOCOL` and `NETTEST_TOKEN` are also honoured, which keeps tokens out of command lines and shell history.

Target shorthand

The first positional argument sets protocol, host and port at once: `wss://edge.example.com:9102`, `udp://10.0.0.5:9100`, `[fd00::5]:9101`, or just `10.0.0.5`. Serverless targets: `ping://10.0.0.77`, `connect://10.0.0.77:5060` (the port is required), `sip://10.0.0.77`, `https://10.0.0.77/admin`. The same forms are used for targets in the monitor's configuration file (section 9). A device literally named `monitor` or `service` needs a scheme (`ping://monitor`) so it is not taken for the subcommand of the same name.

6. Client: headless and scripted use

Add `--no-tui` to run without the interactive view. The client prints events to the console, writes whatever files are enabled, prints a summary at the end, and returns an exit code that scripts can act on. This is the mode for scheduled tasks, cron, remote shells and one-off checks from a script. For continuous watching with alerts, use the monitor (section 9) instead of a loop around headless runs.

```
# Five minutes of 100 ms probes over wss, CSV + JSONL + HTML report into C:\runs
nettest-client --no-tui wss://edge.example.com:9102 --fingerprint 6A:52:9B:... ^
-i 100ms -d 5m --csv --jsonl --report -o C:\runs
# Overnight stability check: heartbeat every 5 s for 8 hours, log every disconnect and
reconnect
nettest-client --no-tui ws://10.0.0.5:9101 -m soak --heartbeat 5s -d 8h --text
# Throughput, 10 seconds each direction
nettest-client --no-tui tcp://10.0.0.5:9100 -m throughput --tp-secs 10
# Scripted check: fail if loss exceeds 1% or p95 exceeds 80 ms, one JSON line on stdout
nettest-client --no-tui udp://10.0.0.5:9100 -i 50ms -c 600 --max-loss 1 --max-p95-ms 80 ^
--json-summary --quiet --no-report
# An IP phone with no server behind it: SIP OPTIONS every 10 s overnight, CSV and report
nettest-client --no-tui sip://10.0.0.77 -m soak --heartbeat 10s -d 8h --csv --report
```

On Linux use `\` instead of `^` for line continuation.

Switch	Purpose
<code>--max-loss <pct></code>	Exit 1 if loss is above this percentage. Default 0, so any loss fails the run.
<code>--max-p95-ms <ms></code>	Exit 1 if the p95 round-trip time is above this value.
<code>--json-summary</code>	Print the final summary as one JSON object on standard output instead of the text table. Everything else goes to standard error.
<code>--quiet</code>	Suppress per-probe and event console output.
<code>--all-samples</code>	Print every probe to the console, not only failures.

Exit codes

Code	Meaning
0	Run completed and loss and p95 were within thresholds.
1	Run completed but loss or p95 exceeded the thresholds.
2	Could not connect, authentication rejected, TLS failed (wrong fingerprint), or a serverless probe could not be set up (name not found, ICMP not permitted).
3	Bad arguments, unreadable configuration, wss without <code>--insecure</code> or <code>--fingerprint</code> , or throughput mode with a serverless protocol.
4	Could not write a log file or report.
5	Interrupted with Ctrl-C before a bounded run (<code>-c</code> or <code>-d</code>) completed.

Ending an unbounded run with Ctrl-C is normal and returns 0 or 1 according to the thresholds.

7. Test modes and what they measure

Mode	What happens	What you get
latency	The client sends numbered probes at a fixed interval; the server echoes each one back.	Round-trip time min/avg/max and p50/p95/p99, jitter, loss percentage, late replies, out-of-order and duplicate counts.
soak	One long-lived connection with periodic heartbeats. If it drops, the client reconnects with increasing back-off and keeps going.	Every disconnect with its reason, reconnect attempts and downtime per outage, uptime percentage, longest outage, longest unbroken session, plus the same RTT statistics from the heartbeats.
throughput	The client streams data to the server for N seconds, then the server streams back for N seconds (or one direction only).	Mbit/s, bytes and frame counts per direction. On UDP, the percentage of datagrams that never arrived.

All modes record the connection phases for every connection attempt: DNS lookup, TCP connect, TLS handshake, WebSocket upgrade and the authentication exchange, each timed separately. Phases that do not apply to the protocol are shown as a dash.

How loss is decided. A probe is counted as lost when no echo has arrived after 2 seconds or ten probe intervals, whichever is longer. If the echo turns up later it is moved from "lost" to "late", so slow links are reported as high latency rather than as loss. (The monitor uses its own per-target `timeout` instead, section 9.)

Failed is not lost. When the far end answers negatively before the timeout (TCP connection refused, ICMP host unreachable) the probe is recorded as `failed` with the reason, instead of waiting out the timeout. Failed probes are included in the loss percentage so thresholds behave the same; the summary shows the split.

Round-trip only. RTT is the time from sending a probe to receiving its echo, measured on the client's own clock. The server adds nothing from its clock, so the two machines never need synchronised time. One-way latency is therefore not reported.

8. Testing devices without a server

Many stability complaints concern devices that cannot run nettest-server: desk phones, conference units, switches, access points, printers, gateways. The client tests them directly with a protocol the device already speaks. Pick the scheme in the Protocol field or the target URL; everything else (interval, duration, soak mode, logs, chart, HTML report, exit codes) works as for a nettest-server target.

Scheme	What is sent	What counts as an answer	Notes
ping	ICMP echo request, size from "Frame size" (default 64 bytes).	The matching echo reply.	No port. Unprivileged on Windows and on Linux systems that allow ping sockets (all common distributions); otherwise run as root or use <code>connect</code> . Some devices rate-limit ICMP; keep the interval at 200 ms or slower.
connect	A TCP handshake to the given port, closed immediately with a normal FIN.	Handshake completed. "Connection refused" (host up, port closed) is reported as <code>failed</code> , distinct from a timeout (host or path down).	Port is required (<code>connect://phone:5060</code>). Works against any open TCP port, including the nettest-server ports.
sip	A SIP <code>OPTIONS</code> request over UDP (default port 5060), the standard keep-alive that phones, PBXs and SBCs answer.	Any final SIP response (200 OK, 401, 405, 486...) whose CSeq and Call-ID match the request. The status line is kept as the sample detail.	Tests the phone's SIP stack, not just its IP. Phones that disable <code>OPTIONS</code> show 100% loss; use <code>connect://phone:5060</code> then. SIP over TCP is not probed.
http / https	A <code>HEAD</code> request for the path (default <code>/</code>) with a <code>Host</code> header, timed to the first line of the response.	Any HTTP status line, including 401 and 404 (the web stack is up). The status line is kept as the sample detail.	<code>https</code> accepts any certificate (devices use self-signed ones) unless a fingerprint is pinned. The RTT includes connect and, for <code>https</code> , the TLS handshake.

Stability without a connection

Serverless probes have no connection to lose, so soak mode derives stability from the probes themselves: when **Outage after N failures** (default 3) consecutive probes fail or go unanswered, an outage opens and is logged with the reason; the next answered probe closes it and the downtime is recorded. Disconnect count, uptime percentage, longest outage and longest unbroken session are then reported exactly as for a nettest-server soak, and the header shows "disconnected" while an outage is open. The heartbeat interval is the probe interval in soak mode.

What is the same, what differs

Same: RTT min/avg/max, percentiles, jitter, loss %, the live chart with loss markers, console/CSV/text/JSONL logs, the HTML report, `--max-loss` / `--max-p95-ms` thresholds and exit codes.

Connection phases show only DNS (and the peer address); there is no TLS, WebSocket or authentication phase to time.

A new status `fail` appears in logs and CSV for refused/unreachable answers, with the reason in the detail column. "Late" cannot occur: an answer after the timeout is discarded.

On Linux an ICMP "destination unreachable" for `ping` is not delivered to unprivileged sockets, so an unreachable host appears as timeouts (as with the `ping` command). Windows reports it as failed.

Throughput mode is not available: there is nothing on the far end to send or receive bulk data.

Examples

```
# Is the phone there, and how is the path to it? 120 pings, one every half second
nettest-client --no-tui ping://10.0.0.77 -i 500ms -c 120
# Is its SIP port reachable through the night? Outage after 2 missed probes
nettest-client --no-tui connect://10.0.0.77:5060 -m soak --heartbeat 5s --outage-after 2 -d 10h
--csv
# Does the SIP stack answer, and how fast? Status lines in the CSV detail column
nettest-client --no-tui sip://10.0.0.77 -c 60 -i 1s --csv --all-samples
# The device's web interface, self-signed certificate accepted
nettest-client --no-tui https://10.0.0.77/ -c 20
# Interactive: pre-filled, then press s
nettest-client sip://10.0.0.77
```

9. Client: monitoring endpoints as a service

`nettest-client monitor` turns the client into an always-on watcher. It probes any number of endpoints continuously, decides when each one is **DOWN** and when it is **UP** again, and sends a message to **ntfy**, **Slack** or **Discord** on every change. Installed as a service it starts at boot and keeps going through reboots, endpoints that are down when it starts, and webhook endpoints that are temporarily unreachable. It uses exactly the probes from sections 7 and 8, so anything you can test you can monitor: a `nettest-server` across a WAN link, a PBX, a gateway, a printer's web interface.

Typical use: one monitor per site, watching the things that site depends on, telling the on-call phone within seconds when the link to head office, the SIP trunk or the gateway stops answering, and again when it recovers, with the time it was down.

Workflow

```
nettest-client monitor --example-config > monitor.toml      # annotated template; edit targets
and webhooks
nettest-client monitor --config monitor.toml --check        # validate, list what would be
watched
nettest-client monitor --config monitor.toml --test-notify  # one test message to every
notifier
nettest-client monitor --config monitor.toml                # run in the foreground (Ctrl-C
stops it)
nettest-client service install --from monitor.toml          # install and start the service
(asks for root / UAC)
nettest-client service status
nettest-client service restart                              # after editing the installed
monitor.toml
nettest-client service uninstall [--purge]                  # --purge also removes monitor.toml
and the log
```

Run `--check` and `--test-notify` before installing: the first catches typos in the file, the second proves the webhook URL, token and TLS path work from this machine. Both exit 0 on success.

The configuration file

`monitor.toml` has one `[monitor]` section, one `[[targets]]` block per endpoint and one `[[notify]]` block per destination. Durations are written with a unit: `500ms`, `10s`, `2m`, `1h30m`; `"0"` disables a timer. A minimal file:

```

[[targets]]
name = "gateway"
target = "ping://10.0.0.1"           # any target form from section 5
interval = "5s"
timeout = "2s"
failures_before_down = 3             # DOWN after 3 consecutive failures (about 15 s here)
successes_before_up = 1              # UP on the first success
remind_every = "1h"                  # repeat the DOWN alert hourly while still down; "0" =
never

[[targets]]
name = "pbx"
target = "sip://pbx.example.com"

[[targets]]
name = "head office nettest"
target = "wss://edge.example.com:9102"
token = "s3cret"
fingerprint = "6A:52:9B:..."       # or insecure = true

[[notify]]
kind = "ntfy"                        # ntfy | slack | discord
url = "https://ntfy.sh/my-secret-topic"
priority = "high"

[[notify]]
kind = "slack"
url = "https://hooks.slack.com/services/T000/B000/XXXX"

```

[monitor] key	Default	Meaning
log_file	(stderr only)	Append the monitor log to this file. The service sets its own path (table below), so this is for foreground runs.
jsonl_file	(off)	The same log as JSON Lines, for log collectors.
summary_every	1h	Write one status line per target (state, how long, probes sent, loss %, p95) at this cadence. "0" turns it off.
notify_on_start	false	Send a "monitor started" message when the monitor starts, so a reboot or a service restart is visible.
hostname	(detected)	Name of this machine as it appears in alert text ("monitor on <hostname>").

[[targets]] key	Default	Meaning
name	the target	Label used in alerts and the log. Must be unique.
target	(required)	What to watch, in the command-line form: ping://host, connect://host:port, sip://host, http://host/path, https://host/, ws://host:9101, wss://host:9102, tcp://host:9100, udp://host:9100.
interval	10s	Time between probes (minimum 100 ms). For ws/wss/tcp/udp the connection stays open and a probe is sent at this cadence.
timeout	5s	Connect timeout and probe timeout. A probe unanswered after this long counts as a failure, whatever the interval.

<code>failures_before_down</code>	3	Consecutive failures that make the target DOWN.
<code>successes_before_up</code>	1	Consecutive successes that make it UP again. Raise it for endpoints that answer once and die again.
<code>remind_every</code>	0	Repeat the DOWN alert at this cadence while the target stays down. "0" sends it once.
<code>token</code> , <code>insecure</code> , <code>fingerprint</code>		As for the client (section 5). A <code>wss://</code> target needs <code>insecure = true</code> or a <code>fingerprint</code> , because nettest-server certificates are self-signed.
<code>ws_path</code> , <code>ipv6</code> , <code>payload_bytes</code>		WebSocket / HTTP path (a path in the target URL also works), <code>true/false</code> to force IPv6/IPv4, probe size (ping and nettest protocols).

[[notify]] key	Default	Meaning
<code>kind</code>	(required)	<code>ntfy</code> , <code>slack</code> or <code>discord</code> .
<code>url</code>	(required)	ntfy: the topic URL (<code>https://ntfy.sh/<topic></code> or your own server). Slack: the incoming-webhook URL. Discord: the channel's webhook URL. Treat these URLs as secrets; anyone who has one can post to the channel.
<code>name</code>	<code>kind#n</code>	Label in the log ("notify ops phone: delivered ...").
<code>token</code>	(none)	ntfy access token for protected topics, sent as <code>Authorization: Bearer</code> .
<code>priority</code>	<code>high</code>	ntfy priority of DOWN alerts (<code>min</code> , <code>low</code> , <code>default</code> , <code>high</code> , <code>urgent</code> or 1-5). UP alerts always use <code>default</code> .
<code>tags_down</code> , <code>tags_up</code>	<code>warning</code> , <code>red_circle</code> / <code>white_check_mark</code>	ntfy tags; emoji short codes are shown as icons by the apps.
<code>insecure</code> , <code>fingerprint</code>	<code>off</code>	For a self-hosted ntfy with a self-signed certificate: accept any certificate, or pin its SHA-256. Public endpoints are verified against the built-in root certificates and need neither.
<code>timeout</code>	<code>10s</code>	Per-attempt timeout for the webhook request.

The alert rule

Every probe is a success or a failure. For `ws`, `wss`, `tcp` and `udp` a lost connection and each reconnect attempt that fails also count as failures, and a successful reconnect as a success. A target becomes **DOWN** after `failures_before_down` consecutive failures and **UP** again after `successes_before_up` consecutive successes. Anything shorter than that never produces a message, so a single dropped ping is invisible by design. The time to an alert is therefore roughly `failures_before_down × interval` plus one `timeout`: with the defaults (`10s`, `5s`, `3`) a dead host is reported about 35 seconds after it stopped answering; with `interval = "2s"`, `timeout = "2s"` it is 8 seconds.

An endpoint that is already down when the monitor starts is reported DOWN after the same number of failures ("already down when the monitor started" in the text). An endpoint that is healthy at start is not announced at all, so a reboot of the monitor machine does not produce a burst of "UP" messages; set `notify_on_start` if you want to hear from it. While a target stays down, `remind_every` repeats the alert with the accumulated downtime.

What the messages look like

```
nettest: gateway DOWN
gateway (ping://10.0.0.1) is DOWN since 2026-10-05 14:03:11 -05:00 after 3 consecutive
failures:
probe lost (no reply) – monitor on branch-pc

nettest: gateway UP
gateway (ping://10.0.0.1) is UP again at 2026-10-05 14:09:40 -05:00 after 6m29s down – monitor
on branch-pc

nettest: pbx still DOWN
pbx (sip://pbx.example.com:5060) has been DOWN for 1h00m: timed out – monitor on branch-pc
```

The first line is the title (ntfy notification title, bold in Slack and Discord); the rest is the body. The reason is the last failure seen: "probe lost (no reply)", "connection refused", "closed by peer", "reconnect attempt 3 failed: ...". All configured notifiers receive every message. Delivery runs in its own task so a slow webhook never delays probing: three attempts per message (after 2 s and 5 s; a 429 answer is retried after the time the service asks for), a 4xx answer other than 429 is treated as permanent (a wrong URL will not fix itself), and every outcome is written to the log.

The log

The monitor writes one line per transition, per delivery result and (every `summary_every`) per target, so the log alone tells the story even if the webhooks were unreachable. In the foreground the log goes to the console; as a service to the file in the table below (and, on Linux, to `journalctl -u nettest-monitor`).

```
2026-10-05 13:09:07.816 INFO nettest-client 1.1.0 monitor starting on branch-pc: 3 target(s),
1 notifier(s)
2026-10-05 13:09:07.818 INFO [gateway] up
2026-10-05 13:09:12.219 WARN [pbx] DOWN after 3 consecutive failures: connection refused
2026-10-05 13:09:12.219 INFO notify ops phone (ntfy): delivered "nettest: pbx DOWN" (200) in
143 ms
2026-10-05 13:09:17.921 INFO [pbx] UP again after 5s
2026-10-05 14:09:07.817 INFO summary gateway: UP for 1h00m sent 720 lost 0 (0.00%) p95 1.2
ms last 0.9 ms
```

Running it as a service

`nettest-client service install --from monitor.toml` validates the file, copies it to the system location, copies the executable, registers the service to start at boot and starts it. The installed file is the only one the service reads; to change anything edit it there (or edit your copy and run `install --from` again) and run `nettest-client service restart`, which checks the file before restarting so a typo does not leave you without monitoring. Administrator rights are obtained exactly as for the server (section 3): `sudo` on Linux, the UAC prompt on Windows, and `status` needs none.

Subcommand	What it does
<code>install --from <file></code> <code>[--no-copy] [--account]</code>	Validate and install the configuration, copy the executable (<code>--no-copy</code> runs it from where it is), register and start. Without <code>--from</code> it re-installs with the configuration already in place (an upgrade).
<code>uninstall [--purge]</code>	Stop and remove the service and the copied executable. <code>--purge</code> also deletes the installed <code>monitor.toml</code> and the log; a <code>nettest-server</code> installed on the same machine keeps its files.
<code>start</code> , <code>stop</code> , <code>restart</code>	Control the installed service. <code>restart</code> re-reads <code>monitor.toml</code> after validating it.

status	Whether the service is installed and running, the targets and notifiers from the installed file, and where the log is.
--------	--

	Linux (systemd)	Windows
Executable	/usr/local/bin/nettest-client	%ProgramFiles%\nettest\nettest-client.exe
Settings	/etc/nettest/monitor.toml (root only, mode 0600; it may hold tokens and webhook URLs)	%ProgramData%\nettest\monitor.toml
Log	/var/log/nettest-monitor/monitor.log and journalctl -u nettest-monitor	%ProgramData%\nettest\monitor.log
Registration	/etc/systemd/system/nettest-monitor.service	Service nettest-monitor ("nettest monitor" in Services), automatic start, depends on Tcpip
Runs as	A dynamic system user with no shell or home, read-only system, no privilege escalation, plus the raw-socket capability so ping:// works whatever the ping-socket policy.	LocalSystem by default; --account as for the server.

The monitor and the server can be installed on the same machine: they share /etc/nettest and %ProgramData%\nettest but no file names, and each uninstall --purge removes only its own files. Upgrading works as for the server: copy the new executable next to the old one and run nettest-client service install with no arguments.

Exit codes of nettest-client monitor: 0 stopped by Ctrl-C or systemctl stop, 2 --test-notify could not deliver to every notifier, 3 the configuration file is missing or invalid (every problem is listed), 4 the log file cannot be written. Service subcommands: 0 ok, 2 service-manager failure, 3 usage, not supported or elevation declined.

10. Exports and log files

Files are written to the output directory (default: the current directory) and named `nettest-<YYYYMMDD-HHMMSS>.<ext>`. Each switch accepts an optional explicit path instead, for example `--csv run1.csv`.

Output	File	Content
Console	screen	Headless only. One line per failed probe plus every event (connect, disconnect, reconnect, result). <code>--all-samples</code> lists every probe; <code>--quiet</code> silences it.
CSV	<code>.csv</code>	One row per probe <i>and</i> per event. Columns: timestamp, elapsed seconds, kind, sequence, status (ok/lost/late/ooo/dup/fail), RTT ms, running loss %, p50, p95, p99, jitter, detail (failure reason, or the SIP/HTTP status line for serverless probes; event text for events). Opens directly in Excel.
Text log	<code>.log</code>	The same lines as the console, honouring "log failures only".
JSON Lines	<code>.jsonl</code>	One JSON object per probe and event, for Splunk, Elastic, Loki and similar.
HTML report	<code>.html</code>	Single self-contained page: summary tiles, RTT chart with loss markers, throughput table, stability events, connection-phase table and the event log. No JavaScript and no external resources, so it opens offline and is safe to attach to a ticket or e-mail.
Server log	any	<code>--log-file</code> and <code>--jsonl</code> on the server record each client's connect, disconnect reason and per-session totals, plus every <code>[scan]</code> banner reply: the server-side half of the evidence. For a service the log is at the location in section 3.
Monitor log	<code>monitor.log</code>	Every DOWN / UP transition with its reason, every webhook delivery or failure, and a periodic status line per target (section 9). <code>jsonl_file</code> gives the same as JSON Lines. The monitor writes no per-probe files.

In the interactive view the HTML report is written automatically when a run ends (if the toggle is on) and can also be written at any moment with `r`. The report path is shown on the status line and the summary screen.

11. Reading the results

Figure	How to read it
last / min / avg / max RTT	Round-trip time in milliseconds. On a LAN expect well under 1 ms; across a WAN tens of ms. Max alone is not alarming; a max far above p99 is a single spike.
p50 / p95 / p99	Percentiles. p95 = 25 ms means 95% of probes returned within 25 ms. These describe what users feel better than the average: a path that is usually fast but stalls once a minute has a fine average and a bad p99.
jitter	Smoothed variation between consecutive RTTs (the estimator used by VoIP). Steady 50 ms is fine for voice; 10 ms that swings by 30 ms is not.
lost / loss %	Probes with no echo. On TCP and WebSocket this is rare because the transport retransmits; loss there shows up as RTT spikes instead. UDP shows true packet loss.
failed	Probes answered negatively (connection refused, host unreachable). Part of the loss count. Many failures with "connection refused" mean the host is up but the service on that port is not.

late	Echoes that arrived after the loss timeout. Many late replies mean a very slow or congested path rather than a lossy one.
ooo / dup	Out-of-order and duplicate echoes. Common on UDP across multi-path links; on TCP they indicate something unusual in the path.
in flight	Probes sent but not yet answered. A climbing value during a run means the path has stalled.
Connect phases	DNS, TCP, TLS, WS upgrade, hello. Whichever is large is where the delay lives: resolver, network, certificate handling, or a proxy.
Soak: disconnects, uptime, longest outage	How often the connection died, how long it stayed dead, and how long it stayed up. Downtime per event is in the HTML report and logs.
Throughput Mbit/s	Payload bits per second on the wire for that direction. Compare upload with download from the same run.
Monitor: DOWN / UP, "after 6m29s down"	The downtime in an UP message is measured from the moment the target was declared DOWN, so it is shorter than the real outage by about the detection time. The periodic summary line gives the loss % and p95 since the monitor started, for the "it is up but bad" cases an alert cannot express.

12. Troubleshooting recipes

Symptom	What to run and what it tells you
Application disconnects every few minutes	Soak over the application's protocol with the heartbeat at, say, 30 s. Disconnects at a regular interval point at a NAT, firewall or load-balancer idle timeout. Lower the heartbeat below that interval; if the drops stop, keep-alives are the fix.
Calls or video are choppy	UDP latency test at 20 ms interval for a few minutes. Look at loss %, jitter and p99. Loss and jitter on UDP are what real-time traffic experiences.
"The network is slow"	Latency test over TCP and over ws. Check the connect phases first: high DNS means the resolver, high TCP means the path, high TLS or WS upgrade means a middlebox or certificate issue. Then look at p95 and p99 rather than the average.
WebSocket app fails but the website loads	Run ws and tcp tests to the same server. TCP fine and ws failing with a TCP time but no WS-upgrade time means a proxy or load balancer refuses the upgrade.
Transfers are slow in one direction	Throughput mode with direction "both". A large asymmetry between upload and download usually means traffic shaping or a duplex mismatch.
Loss on UDP but not on TCP	That is genuine packet loss. TCP hides it by retransmitting, which appears as RTT spikes and a high p99 in the TCP test.
Client exits with code 2 immediately	Wrong port or protocol for the server's listener, firewall blocking the port, token mismatch, or a wss fingerprint that does not match. The console line before the summary says which.
A phone keeps losing registration	Run <code>sip://phone -m soak --heartbeat 10s</code> from the same network segment overnight. Outages that coincide with the drops put the fault on the network or the phone; a clean run points at the PBX or the trunk, which you test with <code>connect://pbx:5060</code> or a nettest-server next to the PBX.
Devices on a switch "disappear" briefly	<code>ping://device -m soak --heartbeat 1s --outage-after 3</code> against two or three devices on that switch at once. Simultaneous outages are the switch or its uplink; a single device alone is that device.
"Tell me when the branch loses the head office"	Install the monitor (section 9) on a machine at the branch with <code>ping://</code> or <code>connect://</code> targets for the head-office gateway and a <code>wss://</code> target for a nettest-server there, plus an ntfy topic on the on-call phone. The DOWN message arrives seconds after the link dies, the UP message says how long it was gone.
Monitor alerts never arrive	<code>nettest-client monitor --config monitor.toml --test-notify</code> on the monitor machine. The log line says exactly what the webhook answered: a 404 or "no_service" is a wrong URL, a 401/403 a bad token, "timed out" a firewall or proxy between the machine and the endpoint, a TLS error a self-hosted ntfy without <code>insecure</code> or <code>fingerprint</code> . As a service, read the monitor log: every delivery attempt is recorded there.
Monitor alerts too often for a flaky but working endpoint	Raise <code>failures_before_down</code> or lengthen <code>interval</code> so a single lost probe cannot trigger; raise <code>successes_before_up</code> so one lucky reply does not end an outage. Use <code>remind_every</code> instead of a short interval if you only want to be nagged while it stays down.
A <code>ping://</code> target is always DOWN in the monitor but <code>nettest-client ping://</code> works	Run the monitor in the foreground as the same user: the log's first lines say why probing cannot start. On Linux the installed service already has the raw-socket capability; on an older kernel or a hardened sysctl use <code>connect://</code> to a known port instead.
<code>ping://</code> reports ICMP not permitted	Linux only: the account's group is outside <code>net.ipv4.ping_group_range</code> . Run as root, widen the range, or use <code>connect://host:port</code> .

A vulnerability scan flags an unknown service on 9100-9102	That is nettest-server; open <code>http://host:9100/</code> in a browser to see the identification banner. The text is configurable with <code>--banner</code> if the scanner needs a particular string.
Need evidence for a carrier or vendor	Run headless with <code>--csv --report</code> for the agreed period and attach the HTML report; it contains the chart, every event, and the per-phase timings in one file. For an outage history over weeks, the monitor log lists every DOWN and UP with timestamps and durations.

13. Quick reference card

Server

```
nettest-server                                # interactive view,
default ports
nettest-server --no-tui --token <secret> --log-file server.log    # console logging,
token required
nettest-server --bind <ip> --ws-port 9100 --tcp-port 0 --udp-port 0 --wss-port 0  # one
listener only
nettest-server --save-config                    # make current switches
the defaults
nettest-server service install --generate-token [--ws-port N ...] # permanent service
(token required)
nettest-server service edit --idle-timeout 120                    # change a setting,
restart
nettest-server service status | start | stop | restart | uninstall # manage it
```

Default ports: tcp 9100, udp 9100, ws 9101, wss 9102. Port 0 disables a listener. Every port answers scanners with the banner.

Client

```
nettest-client ws://<host>:9101                # interactive, pre-
filled target
nettest-client --no-tui tcp://<host>:9100 -i 200ms -c 300 --csv    # 300 probes, CSV +
HTML report
nettest-client --no-tui ws://<host>:9101 -m soak --heartbeat 5s -d 8h # overnight stability
nettest-client --no-tui tcp://<host>:9100 -m throughput --tp-secs 10 # bandwidth both
directions
nettest-client --no-tui wss://<host>:9102 --fingerprint <sha256>   # TLS with pinned
certificate
nettest-client --no-tui ping://<device> -c 100                    # no server: ICMP
nettest-client --no-tui sip://<phone> -m soak --heartbeat 10s -d 8h # no server: SIP
OPTIONS stability
nettest-client --no-tui connect://<device>:<port> -m soak          # no server: TCP port
reachability
```

Monitor

```
nettest-client monitor --example-config > monitor.toml            # template with every
key explained
nettest-client monitor --config monitor.toml --check              # validate
nettest-client monitor --config monitor.toml --test-notify        # test the webhooks
nettest-client monitor --config monitor.toml                      # foreground
nettest-client service install --from monitor.toml                # permanent service
nettest-client service status | restart | stop | start | uninstall # manage it
```

Alert rule: DOWN after `failures_before_down` (3) consecutive failures, UP after `successes_before_up` (1) successes, optional `remind_every`. Notifiers: `ntfy`, `slack`, `discord`.

Interactive keys	
s	start / run again
x	stop
r	write HTML report
w	save settings
1 2 3	chart window 60 s / 5 min / all
c	clear log
?	help
q	quit

Exit codes	
0	ok
1	thresholds exceeded
2	connect / auth / TLS / webhook test failed
3	usage / configuration
4	file write failed
5	interrupted

Files

Settings: %APPDATA%\nettest\ or ~/.config/nettest/

Server service: %ProgramData%\nettest\ or /etc/nettest/server.toml, /var/lib/nettest/, /var/log/nettest/

Monitor service: %ProgramData%\nettest\monitor.toml or /etc/nettest/monitor.toml, /var/log/nettest-monitor/

Outputs: nettest-<timestamp>.csv / .log / .jsonl / .html